REPORT

Reviewer report: Delgan/loguru#1515: Evaluate every

rotation condition so a skipped one cannot fire spuriously

Run loguru-1515, tested 2026-11-06 14:36 IST by ops/daily_run.py (route source-pair).
Proof-of-Change commit

27fde0369adc43f436734b1573c4014457dad886. Report generated 2026-11-06 14:46 IST by

reports/pr_report.py.

Baseline: full (test comparison + claim probes + coverage gate + claim flagging (differential probes shown,

informational)).

NEEDS ATTENTION

Recommended action: Reject Confidence: High

The run is trustworthy and found a difference a person should look at. Every check passed and every claim cites

a log line.

Advice only: Proof-of-Change never blocks a merge and never posts on GitHub.

Summary in plain English

• What it says it does: Evaluate every rotation condition so a skipped one cannot fire spuriously

(Delgan/loguru#1515).

• What we observed: probe: 1 observation(s) need attention: c2.ret_msg2 differs although the claim says it does

not (C2); probe: 5 unchanged sub-check(s) of an input where the claim's change was seen, shown as advisory

only: c1.cond0_limit_after_msg1 (C1); c1.cond0_limit_after_msg2 (C1); c1.cond1_initialized_after_msg1 (C1);

c1.cond1_initialized_after_msg2 (C1); probe: 2 observation(s) shown as advisory only (claim not shown on these

inputs; a related change was proven): c3.cond1_limit_after_msg1_in_past (C3);

c3.cond1_limit_after_msg2_in_past (C3); probe: claim(s) C1, C2, C4 proven; tests: 1 test(s) fail on main and

pass on the PR; 148 test(s) ran on main, 148 on the PR; 0 new test(s) pass, 150 unchanged (3 of them skipped

on both sides).

• What you should do: Do not approve as is. Read 'What changed in behavior' and decide with the PR author.

Demo helper

These notes are background from the console and the daily check. They do not change the verdict.

If setup fails, the helper retries the install up to 2 times. If a test still can't be verified because something is

missing or fails to import, it looks up the cause, installs the fix on both sides and reruns that step, up to 3 tries.

• Setup check: GitHub token: present.

• Setup check: Nebius token: present.

• Setup check: free memory 9.8 GB (of 15.6 GB).

• Setup check: free disk 94.2 GB.

• Demo helper: git is installed.

• Demo helper: python3 is installed.

• Demo helper: uv is installed.

• Looked up with Tavily: [redacted] results

• Query: loguru python setup install instructions https://github.com/Delgan/loguru site:github.com/Delgan/loguru

• Query: loguru python documentation install https://github.com/Delgan/loguru site:github.com/Delgan/loguru

• Query: loguru python install requirements https://pypi.org/project/loguru/ https://github.com/Delgan/loguru

site:pypi.org

• Source: loguru/pyproject.toml at master · Delgan/loguru · GitHub —

https://github.com/Delgan/loguru/blob/master/pyproject.toml

• Source: lager · PyPI — https://pypi.org/project/lager/

• Source: Python Logging (beans-logging) - PyPI — https://pypi.org/project/beans-logging/

• Source: loguru - PyPI — https://pypi.org/project/loguru/

• Source: loguru/.github/workflows/tests.yml at 0d1355c8a4c75db5ded27a3b5e9ce9d0e66b2d47 · Delgan/loguru

· GitHub — https://github.com/Delgan/loguru/blob/0d1355c8a4c75db5ded27a3b5e9ce9d0e66b2d47/.github/work

flows/tests.yml

• Source: loguru/.github/workflows/tests.yml at 1315750610b108129d2fa432230a9a92c608cbcd · Delgan/loguru

· GitHub — https://github.com/Delgan/loguru/blob/1315750610b108129d2fa432230a9a92c608cbcd/.github/workf

lows/tests.yml

• Source: loguru/.github/workflows/tests.yml at f3dc4c6622e0c2ce978829fbdce13bbfb76862e8 · Delgan/loguru ·

GitHub — https://github.com/Delgan/loguru/blob/f3dc4c6622e0c2ce978829fbdce13bbfb76862e8/.github/workflo

ws/tests.yml

• Source: loguru/.readthedocs.yml at master · Delgan/loguru · GitHub —

https://github.com/Delgan/loguru/blob/master/.readthedocs.yml

• Source: Bump version to 0.7.0 · Delgan/loguru@9fc929a · GitHub —

https://github.com/Delgan/loguru/commit/9fc929a54ae6ef92208c90ebafd449b8f9e3e34a

• Setup plan: the same packages on main and pr.

• Setup plan: colorama is noted, installed with the project.

• Setup plan: aiocontextvars is noted, installed with the project.

• Setup plan: win32-setctime is noted, installed with the project.

• Setup plan: install pre-commit==4.6.0 ; python_version>='3.11' (listed in the dev extra) on main and pr.

• Setup plan: install tox==3.27.1 ; python_version<'3.8' (listed in the dev extra) on main and pr.

• Setup plan: install pytest==6.1.2 ; python_version<'3.8' (listed in the dev extra) on main and pr.

• Setup plan: install pytest-cov==2.12.1 ; python_version<'3.8' (listed in the dev extra) on main and pr.

• Setup plan: install pytest-mypy-plugins==4.0.3 ; python_version>='3.12' (listed in the dev extra) on main and pr.

• Setup plan: install freezegun==1.1.0 ; python_version<'3.8' (listed in the dev extra) on main and pr.

• Setup plan: install exceptiongroup==1.1.3 ; python_version>='3.7' and python_version<'3.11' (listed in the dev

extra) on main and pr.

• Setup plan: install mypy==v0.910 ; python_version<'3.6' (listed in the dev extra) on main and pr.

• Setup plan: install Sphinx==8.2.1 ; python_version>='3.11' (listed in the dev extra) on main and pr.

• Setup plan: install sphinx-rtd-theme==3.0.2 ; python_version>='3.11' (listed in the dev extra) on main and pr.

• Setup plan: install myst-parser==4.0.0 ; python_version>='3.11' (listed in the dev extra) on main and pr.

• Setup plan: install build==1.3.0 ; python_version>='3.11' (listed in the dev extra) on main and pr.

• Setup plan: install twine==6.2.0 ; python_version>='3.11' (listed in the dev extra) on main and pr.

• Setup plan: Python 3.12: Python 3.12 already meets >=3.5, <4.0,>=3.5

• Setup plan: Python rule: one Python version is used on both sides: the lowest version that meets the

requirements of both main and the pull request; if none meets both, the check stops as a setup problem and

records both.

• Setup plan: refused pip: its PyPI page does not link to this GitHub repository

• Repository crawl: 9 page(s) crawled, 8 read in full, 1 setup step(s) used first in the sandbox (3 Tavily call(s), 5.0

credit(s)).

Claims vs reality

| # | Claim (source) | Status | What we observed | Log line

| C2 | Existing behavior is kept: tests that pass on main still pass (implied by every PR) | proven | no test that

passes on main fails on the PR (150 unchanged, 3 runs per side) | pytest-pr-1.log:8 «148 passed, 3 skipped in

26.05s»

| P-C1 | After the change, the __call__ method iterates through all rotation conditions and invokes each one,

regardless of earlier results. (PR description) | not observed | probe model-p1, c1.cond0_limit_after_msg1: same

on both sides: "2020-01-02T13:00:00" | probe-model-p1-pr-1.jsonl:1 «"observation":

"c1.cond0_limit_after_msg1"»

| P-C2 | The method returns True exactly when at least one rotation condition evaluates to True, preserving the

documented 'any of which can trigger the rotation' contract. (PR description) | false | probe model-p1,

c2.ret_msg2: value: true -> false | probe-model-p1-pr-1.jsonl:8 «"observation": "c2.ret_msg2"»

| P-C3 | When all rotation conditions are evaluated, a condition that would have been skipped does not retain a

stale limit, thus it does not cause an extra rotation on the next message. (PR description) | not observed | probe

model-p1, c3.cond1_limit_after_msg1_in_past: same on both sides: null | probe-model-p1-pr-1.jsonl:9

«"observation": "c3.cond1_limit_after_msg1_in_past"»

| P-C4 | The test asserts that after two messages only two log files exist; before the change it fails because three

files are created, after the change it passes. (PR description) | proven | probe model-p1, c4.num_true_returns:

value: 2 -> 1 | probe-model-p1-pr-1.jsonl:11 «"observation": "c4.num_true_returns"»

Citation check (PASS): 5 citation(s), every one re-read from its log file and matched.

What changed in behavior

| What | Count | Examples

| Tests that actually ran (main / PR, fewest of the runs) | 148 / 148 | | Tests unchanged | 150 | | New tests that pass | 0 | | Tests that pass on main but fail on the PR | 0 | | Tests that are new in the PR and fail | 0 | | Tests that ran on main but are missing on the PR side | 0 | | Tests that changed result in another way (for example passed -> skipped) | 0 | | Tests that gave different results between the 3 runs of the same side (flaky) | 0 | | Tests that fail on main and pass on the PR | 1 |

tests.test_filesink_rotation::test_multiple_rotation_conditions_all_evaluated

| Tests that are new or changed but did not run because an optional module is missing | 0 | | Tests that were re-parametrized (same test, new parameter ids; not counted as removed) | 0 | | Test modules that fail to load on the PR side only (their tests no longer run) | 0 | | Test modules that fail to load on main only (no baseline for their tests) | 0 | | Test modules that fail to load on both sides (their tests did not run) | 0 | | Tests that fail identically on both sides (already broken on main) | 0 | -

Probes

Model-written probes: model provider: nebius (Token Factory, live).

Probe test settings: read from main's checkout (pyproject.toml), the same on both sides. Variables: none.

A probe is a small deterministic script run on both sides in the same environments as the tests, 3 alternating runs

per side. It prints JSON observations; plain code (poc/diff.py) compares them. A difference that no claim of the

PR covers needs attention. Every probe was frozen (content sha256) before any PR-side code ran, and checked

against that freeze before each PR-side run. Only a live model probe is blind; fixed probes and recorded model

fixtures are known-answer cases.

model-p1 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-p1.py, frozen c3095a854684

at 2026-11-06T09:09:14.532775+00:00): Tests the rotation group behavior for claims C1, C2, C3, and C4.

• P-C1 (PR description): After the change, the __call__ method iterates through all rotation conditions and

invokes each one, regardless of earlier results. Expects the covered observations to differ.

• P-C2 (PR description): The method returns True exactly when at least one rotation condition evaluates to True,

preserving the documented 'any of which can trigger the rotation' contract. Expects the covered observations to

stay the same.

• P-C3 (PR description): When all rotation conditions are evaluated, a condition that would have been skipped

does not retain a stale limit, thus it does not cause an extra rotation on the next message. Expects the covered

observations to differ.

• P-C4 (PR description): The test asserts that after two messages only two log files exist; before the change it

fails because three files are created, after the change it passes. Expects the covered observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c1.cond0_limit_after_msg1 | P-C1 | no change although the claim says it changes | same on both sides:

"2020-01-02T13:00:00" | probe-model-p1-main-1.jsonl:1 «"observation":

"c1.cond0_limit_after_msg1"»
probe-model-p1-pr-1.jsonl:1 «"observation": "c1.cond0_limit_after_msg1"»

| c1.cond0_limit_after_msg2 | P-C1 | no change although the claim says it changes | same on both sides:

"2020-01-02T13:00:00" | probe-model-p1-main-1.jsonl:2 «"observation":

"c1.cond0_limit_after_msg2"»
probe-model-p1-pr-1.jsonl:2 «"observation": "c1.cond0_limit_after_msg2"»

| c1.cond1_initialized_after_msg1 | P-C1 | no change although the claim says it changes | same on both sides:

false | probe-model-p1-main-1.jsonl:3 «"observation":

"c1.cond1_initialized_after_msg1"»
probe-model-p1-pr-1.jsonl:3 «"observation":

"c1.cond1_initialized_after_msg1"»

| c1.cond1_initialized_after_msg2 | P-C1 | no change although the claim says it changes | same on both sides:

false | probe-model-p1-main-1.jsonl:4 «"observation":

"c1.cond1_initialized_after_msg2"»
probe-model-p1-pr-1.jsonl:4 «"observation":

"c1.cond1_initialized_after_msg2"»

| c1.cond1_limit_after_msg1 | P-C1 | as claimed | value: null -> "<timestamp>" | probe-model-p1-main-1.jsonl:5

«"observation": "c1.cond1_limit_after_msg1"»
probe-model-p1-pr-1.jsonl:5 «"observation":

"c1.cond1_limit_after_msg1"»

| c1.cond1_limit_after_msg2 | P-C1 | no change although the claim says it changes | same on both sides:

"2020-01-02T12:00:00" | probe-model-p1-main-1.jsonl:6 «"observation":

"c1.cond1_limit_after_msg2"»
probe-model-p1-pr-1.jsonl:6 «"observation": "c1.cond1_limit_after_msg2"»

| c2.ret_msg1 | P-C2 | as claimed | same on both sides: true | probe-model-p1-main-1.jsonl:7 «"observation":

"c2.ret_msg1"»
probe-model-p1-pr-1.jsonl:7 «"observation": "c2.ret_msg1"»

| c2.ret_msg2 | P-C2 | contradicts the claim | value: true -> false | probe-model-p1-main-1.jsonl:8 «"observation":

"c2.ret_msg2"»
probe-model-p1-pr-1.jsonl:8 «"observation": "c2.ret_msg2"»

| c3.cond1_limit_after_msg1_in_past | P-C3 | no change although the claim says it changes | same on both

sides: null | probe-model-p1-main-1.jsonl:9 «"observation":

"c3.cond1_limit_after_msg1_in_past"»
probe-model-p1-pr-1.jsonl:9 «"observation":

"c3.cond1_limit_after_msg1_in_past"»

| c3.cond1_limit_after_msg2_in_past | P-C3 | no change although the claim says it changes | same on both

sides: null | probe-model-p1-main-1.jsonl:10 «"observation":

"c3.cond1_limit_after_msg2_in_past"»
probe-model-p1-pr-1.jsonl:10 «"observation":

"c3.cond1_limit_after_msg2_in_past"»

| c4.num_true_returns | P-C4 | as claimed | value: 2 -> 1 | probe-model-p1-main-1.jsonl:11 «"observation":

"c4.num_true_returns"»
probe-model-p1-pr-1.jsonl:11 «"observation": "c4.num_true_returns"»

Citation check (PASS): 22 citation(s), every one re-read from its log file and matched.

Was the changed code exercised?

Coverage: MEASURED by coverage.py on the PR side (33.5 s run).

Coverage gate (python, confidence normal): 5 of 5 changed executable line(s) ran (100%).

| Changed file | Changed executable lines | Of those, ran

| loguru/_file_sink.py | 5 | 5

How this is measured: one extra, untimed run of the PR side's tests under coverage.py. Changed lines come

from comparing the tested main and PR files. Compiled code (C, C++, Cython) is invisible to coverage.py; such a

change only counts as verified when the PR adds or changes a test for that module.

What went right

• Tested 6c91de2c42d3 of the PR against main (source-pair route).

• 3 alternating runs per side: main 3, PR 3 of 3.

• pytest exit codes explained: all runs exited 0.

• Changed code was exercised (coverage gate): 5 of 5 changed executable line(s) ran (100%).

• Probes ran on both sides: 1 probe(s), 3 alternating runs per side, every run printed its observations.

• Single-thread environment: OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0.

• Machine type recorded: local x86_64 8 cpus.

• Nothing posted upstream: GitHub was only read.

• Every claim cites a real log line: 5 citation(s), every one re-read from its log file and matched.

• Every probe observation cites its output line: 22 citation(s), every one re-read from its log file and matched.

What went wrong and how to fix it

| Issue | Status | Detail | How to fix

| Behavior difference found | finding | probe: 1 observation(s) need attention: c2.ret_msg2 differs although the

claim says it does not (C2); probe: 5 unchanged sub-check(s) of an input where the claim's change was seen,

shown as advisory only: c1.cond0_limit_after_msg1 (C1); c1.cond0_limit_after_msg2 (C1);

c1.cond1_initialized_after_msg1 (C1); c1.cond1_initialized_after_msg2 (C1); probe: 2 observation(s) shown as

advisory only (claim not shown on these inputs; a related change was proven):

c3.cond1_limit_after_msg1_in_past (C3); c3.cond1_limit_after_msg2_in_past (C3); probe: claim(s) C1, C2, C4

proven; tests: 1 test(s) fail on main and pass on the PR; 148 test(s) ran on main, 148 on the PR; 0 new test(s)

pass, 150 unchanged (3 of them skipped on both sides) | This is about the PR, not the run: read the failing tests

and their log lines and the Probes section, then decide.

Evals and guardrails

| Check | Result | Detail

| 3 alternating runs per side | PASS | main 3, PR 3 of 3

| pytest exit codes explained | PASS | all runs exited 0

| Changed code was exercised (coverage gate) | PASS | 5 of 5 changed executable line(s) ran (100%)

| Probes ran on both sides | PASS | 1 probe(s), 3 alternating runs per side, every run printed its observations

| Single-thread environment | PASS | OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0

| Machine type recorded | PASS | local x86_64 8 cpus

| Nothing posted upstream | PASS | GitHub was only read

| Known-answer replay (fresh re-run) | SKIP | not replayed yet

| Every claim cites a real log line | PASS | 5 citation(s), every one re-read from its log file and matched

| Every probe observation cites its output line | PASS | 22 citation(s), every one re-read from its log file and

matched

Checklist

• I read the verdict and the plain-English summary.

• Every 'false' or 'unmentioned change' row has a log line I can open.

• Nothing under 'What went wrong' is a run problem (those mean Re-run, not Reject).

• I recorded my decision (see below).

Decision recorded: none yet. To record one, run make decide

REPORT=poc-219-live/reports/daily/2026-11-06-219-27fde03-v2/loguru-1515 DECISION=approve

(or reject, approve_with_notes, re_run).

How long each step took

| Step | Seconds
| sandbox prep (checkout) | 15.1
| Tavily setup lookup | 45.2
| demo_helper | 0.0
| Python setup | 0.0
| dependency install | 0.0
| model: claims/probe writing | 12.2
| probes | 119.4

| main side install | 72.1
| tests (both sides) | 304.9
| verdict | 0.0
| record | 0.0
| tests (main side) | 89.6
| tests (PR side) | 89.9
| model call 1 (claims) | 12.2
| model call 2 (probes) | 92.0

Rerun command
make check PR=https://github.com/Delgan/loguru/pull/1515

Evidence links
• Pull request Delgan/loguru#1515
• record.json
• pr.diff
• git.log
• pytest-main-1.log
• pytest-pr-1.log
• pytest-main-2.log
• pytest-pr-2.log
• pytest-main-3.log
• pytest-pr-3.log
• junit-main-1.xml
• junit-pr-1.xml
• coverage-pytest.log
• coverage-pr.json
• probe-model-p1-main-1.jsonl

• probe-model-p1-pr-1.jsonl

• main run 1 result: pytest-main-1.log:8 «147 passed, 3 skipped in 26.03s»

• pr run 1 result: pytest-pr-1.log:8 «148 passed, 3 skipped in 26.05s»

Glossary

| Term | Meaning

| main | The library as it is today, without the PR.

| PR | Pull request: a proposed change to the library.

| run | One full pass of the selected tests. We do 3 per side, alternating main and PR.

| flaky | A test that gives different results when nothing changed; it is ignored as evidence.

| log line | A line in a pytest-*.log file: the raw output the verdict is based on.

| route | How the two sides were built: nightly+patch, source-build or source-pair (see docs/OPERATIONS.md).

| changed lines covered | How many of the PR's changed, executable Python lines ran during the PR side's tests.

| re-parametrized | A test whose parameter ids changed (renamed cases); the new ids are judged on their own.

rebuilt from cleaned text

Written 2026-11-06 14:46 IST